

Guía práctica para hacer commits profesionales

Este documento define **cómo escribir commits claros, útiles y profesionales** en proyectos Front-End (y en general). No es dogma. Es criterio aplicado.

Objetivo de un buen commit

Un commit sirve para que: - otra persona entienda **qué cambió** sin leer todo el código - tú mismo entiendas **por qué tomaste esa decisión** dentro de 6 meses - el historial del proyecto sea **auditable y confiable**

Un buen commit **explica la intención**, no el detalle técnico.

Estructura recomendada

```
<tipo>: <acción clara y concreta>
```

Opcionalmente, puedes agregar un cuerpo:

```
<tipo>: <acción clara y concreta>

- Detalle relevante 1
- Detalle relevante 2
```

Tipos de commit (cuándo usar cada uno)

feat

Cuándo usarlo: - Agregas una funcionalidad nueva - El usuario puede hacer algo que antes no podía

Ejemplos:

```
feat: add portfolio hover interaction
feat: implement responsive navigation menu
```

fix

Cuándo usarlo: - Corrige un bug - Algo funcionaba mal y ahora funciona bien

Ejemplos:

```
fix: prevent overlay overflow on mobile  
fix: correct button focus state
```

refactor

Cuándo usarlo: - Cambias estructura o código interno - NO agregas funcionalidades nuevas - Mejoras legibilidad, arquitectura o mantenimiento

Ejemplos:

```
refactor: improve component architecture and BEM consistency  
refactor: decouple layout from reusable components
```

style

Cuándo usarlo: - Cambios SOLO visuales - No afectan lógica ni comportamiento

Ejemplos:

```
style: adjust spacing and font sizes  
style: update button styles
```

chore

Cuándo usarlo: - Tareas de mantenimiento - No afectan directamente al producto

Ejemplos:

```
chore: clean unused assets  
chore: update project structure
```

docs

Cuándo usarlo: - Cambios en documentación - README, comentarios, guías

Ejemplos:

```
docs: add project architecture explanation
docs: update commit guidelines
```

test

Cuándo usarlo: - Agregas o modificas tests

Ejemplos:

```
test: add unit tests for CTA component
test: improve coverage for utils
```

Qué evitar

 Commits vagos:

```
update
fix stuff
final version
```

 Commits mezclados: - No mezcles refactor + feature en el mismo commit - Si son cosas distintas, separa commits

Reglas prácticas

- Un commit = una intención
- Si dudas entre `feat` y `refactor`, pregúntate:

¿El usuario final nota este cambio?

- Prefiere commits pequeños y claros
- El historial debe contar una historia lógica

Ejemplo real (como el de tu proyecto)

refactor: improve component architecture, BEM consistency and accessibility

- Decoupled layout and reusable components
- Fixed BEM naming and responsibilities
- Improved mobile interactions and focus states

Cierre

Un buen commit **no impresiona**, pero **inspira confianza**. Eso es lo que buscan equipos y reclutadores.

Usa esta guía como estándar base y ajústala según el proyecto.